MLOps II · Trabajo integrador

Plataforma de predicción de ACV

Antonella Gambarte

Un XGBoost estima si un paciente tiene riesgo de ACV



Dataset: Stroke Prediction Dataset de Kaggle.

Arquitectura de la plataforma



Infraestructura: servicios contenerizados y orquestados con Docker Compose.

Un modelo nuevo pasa a producción solo si supera al champion en F1

1

ETL

Del CSV crudo a features; deja medianas y escalado en data.json

2

Entrena

XGBoost, con sus métricas en MLflow

3

Registra

Versión nueva con el alias challenger

4

Compara F1

Si supera al champion lo reemplaza; si empata, no

5

Recarga

FastAPI, gRPC y el worker.

Sesión 1 · REST

Una API REST recibe al paciente y devuelve su riesgo

Qué hace

- POST /v1/predict: entran las 10 variables, sale la clase con su probabilidad
- Pydantic valida antes de predecir: un dato fuera de rango vuelve con 422
- /health avisa si el modelo está cargado

Qué se creó

`api/main.py`
Endpoints de predicción y health

`api/contrato.py`
Contrato Pydantic de entrada y salida

`api/modelo.py`
Carga el champion y lo actualiza

`api/src/predict.py`
Repite el ETL sobre cada paciente

GraphQL devuelve solo los campos que el cliente pide

Qué hace

- Query model: nombre, versión y métricas del champion.
- Las métricas se piden a MLflow solo si la consulta las incluye.
- Mutation predecir: el mismo paciente que REST, validado con el mismo contrato.

Qué se creó

`api/esquema_graphql.py`
Tipos, Query y Mutation con Strawberry

`api/main.py`
Monta /graphql en la misma app

gRPC predice a partir de un contrato tipado en el .proto

Qué hace

- Servicio aparte, y con mensajes binarios (Protocol Buffers)
- Predecir (unary): un paciente, una respuesta
- PredecirLote (server-streaming): las predicciones de un lote llegan de a una

Qué se creó

`grpc_service/scoring.proto`
Contrato: mensajes, enums y servicio

`grpc_service/server.py`
Implementa los dos métodos

`grpc_service/mapas.py`
Pasa los enums a los textos del modelo

Sesión 4 · Streaming

Un worker puntúa cada paciente que llega y avisa si hay drift

Qué hace

- Redpanda recibe los eventos en el topic pacientes
- El productor manda pacientes del lake; desde la mitad, les sube la glucosa
- El worker predice cada evento y mide throughput, p95 y glucosa por ventana
- Si la glucosa media pasa el umbral, publica una alerta en el topic alertas

Qué se creó

`streaming/productor.py`
Simula pacientes y el drift

`streaming/worker.py`
Predice, alerta y guarda en Parquet

MinIO guarda datos y modelos por zonas, como un S3 local

Zonas del bucket data

- **raw/**: CSV original.
- **staged/**: sin duplicados y con one-hot.
- **curated/**: train y test listos para entrenar.
- **models/**: un modelo por versión.
- **data_info/data.json**: contiene las columnas, las medianas de imputación y parámetros de escalado.

Qué se creó

`docker-compose.yaml`
MinIO y los buckets data y mlflow

`airflow/dags/etl_acv.py`
Llena raw, staged y curated

`airflow/dags/entrenar_acv.py`
Publica cada versión en models/

`streaming/worker.py`
Deja los eventos en raw/, en Parquet

Sesión 7 · Seguridad

Un endpoint aparte exige token y rol, limita consultas y audita

Qué controla `/v1/secure/predict`

401 **Token JWT**
Falta, no es válido o venció

403 **Rol**
No es analista ni admin

429 **Límite**
Más de 5 consultas por minuto

200 **Predicción**
Queda auditada con un hash

Qué se creó

`api/seguridad.py`
Token, roles, límite y auditoría

`api/main.py`
Monta la puerta y restringe el CORS

`frontend/server/api/token.post.ts`
Pide el token sin exponer la API key

Demo

1. **Airflow:** corren etl_acv y entrenar_acv
2. **MLflow:** la versión nueva y el alias champion
3. **Frontend:** el mismo paciente por REST, GraphQL y gRPC, y la latencia de cada uno
4. **Puerta protegida:** 401, 403 y 429
5. **Streaming:** el worker detecta el drift de glucosa y avisa